deferring less critical operations. For example, in an input-heavy search component, useTransition ensures typing remains responsive even when filtering a large dataset in the background.

Listing 4.4, located in `components/search-component/SearchComponent.tsx`, demonstrates how to leverage the `useTransition` hook to prioritize user interactions, such as typing, over non-urgent updates like filtering.

Listing 4.4 useTransition example

```
const SearchComponent: React.FC<SearchComponentProps> = ({ items
  const [query, setQuery] = useState<string>(''); #A
  const [results, setResults] = useState<string[]>(items); #B
  const [isPending, startTransition] = useTransition(); #C
  const handleSearch = (e: React.ChangeEvent<HTMLInputElement>) =
    const value = e.target.value;
    setQuery(value); #D
    startTransition(async () => { #E
      await new Promise((resolve) => setTimeout(() => {
        const filteredResults = items.filter((item) =>
          item.toLowerCase().includes(value.toLowerCase())
        );
        setResults(filteredResults);
        resolve();
      }, 1000));
    });
  };
  return (
    <div>
      <input
        type="text"
        value={query}
        onChange={handleSearch}
        placeholder="Search items..."
      />
      {isPending && <p>Loading...</p>}
      <ul>
        {results.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
};
```